Image Transformations

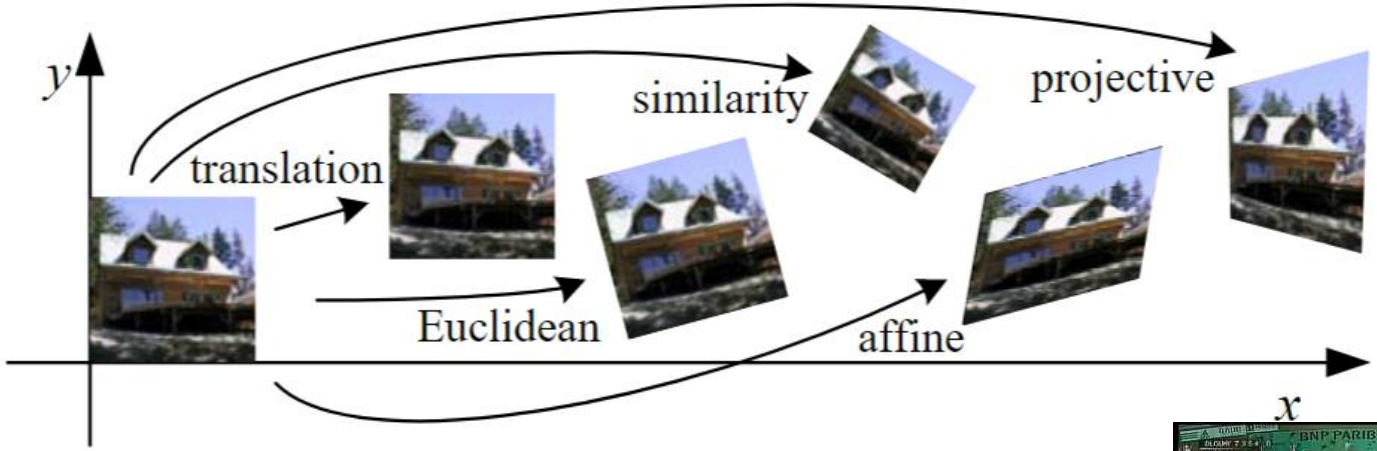
Prof. Jungong Han (jungong.han@sheffield.ac.uk)
Department of Computer Science
The University of Sheffield

Outline

- Geometric transformations
- Interpolation
- Estimating a transformation matrix using algebra
- Practical applications

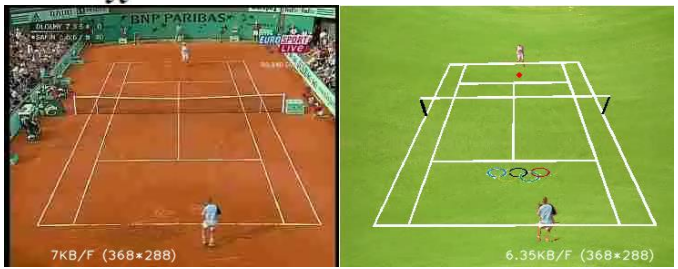
Geometric (spatial) transformations

- Mapping 2D images to new coordinates by a transformation matrix T



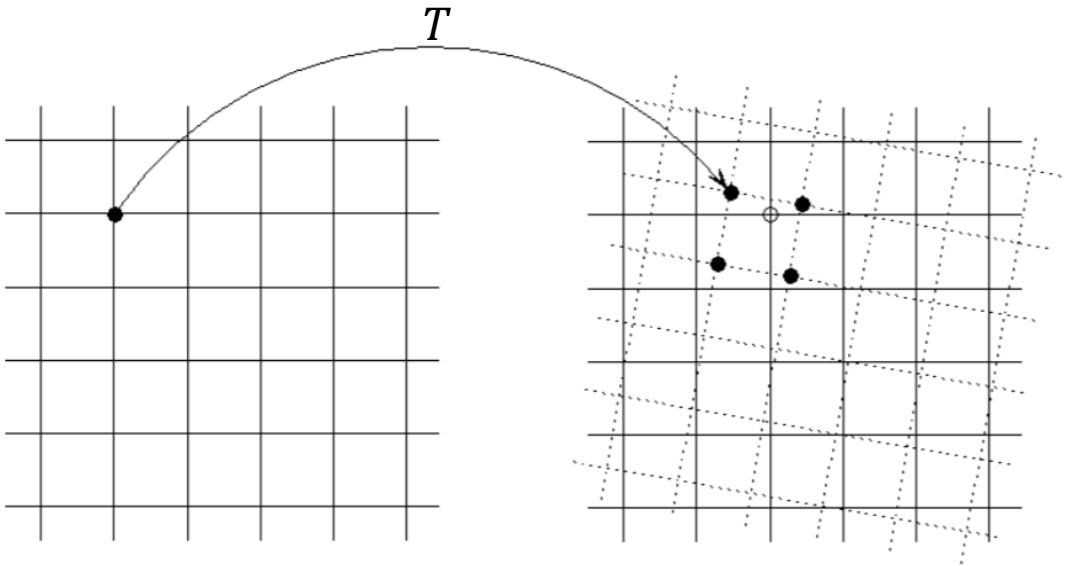
Credit: Szeliski

(Computer Vision: Algorithms and Applications- Section 3.6)



Geometric (spatial) transformations

- Mapping 2D images to new coordinates by a transformation matrix T



Basic transformations

- Identity
- Scale
- Translation
- Rotation

Identity

- Don’t forget that a 2D image I is a set of pixels with two coordinates (x,y) $\in \Omega_I$

- Algebraically :

For each (x,y) $\in \Omega_I$

$\acute{x} = x$

$\acute{y} = y$

$I(\acute{x}, \acute{y}) = I(x, y)$

As matrix multiplication:

$$\begin{bmatrix} \acute{x} \\ \acute{y} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}}_{T_{identity}} \begin{bmatrix} x \\ y \end{bmatrix}$$

Scale

- Algebraically

For each (x,y) $\in \Omega_I$

$$\acute{x} = S_x\, x$$

$$\acute{y} = S_y\, y$$

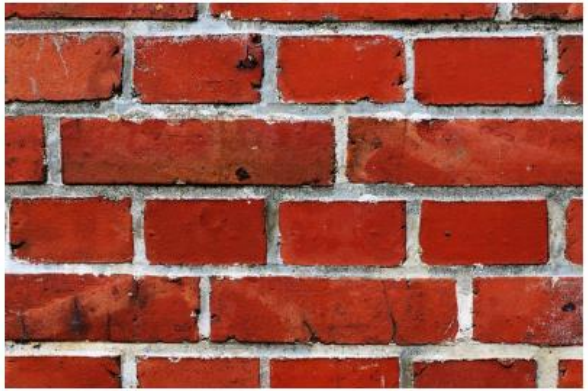
As matrix multiplication:

$$\begin{bmatrix} \acute{x} \\ \acute{y} \end{bmatrix} = \underbrace{\begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix}}_{T_{scale}} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$I(\acute{x}, \acute{y}) = I(x, y)$$

Scaling

Original



$s_x = 0.5$

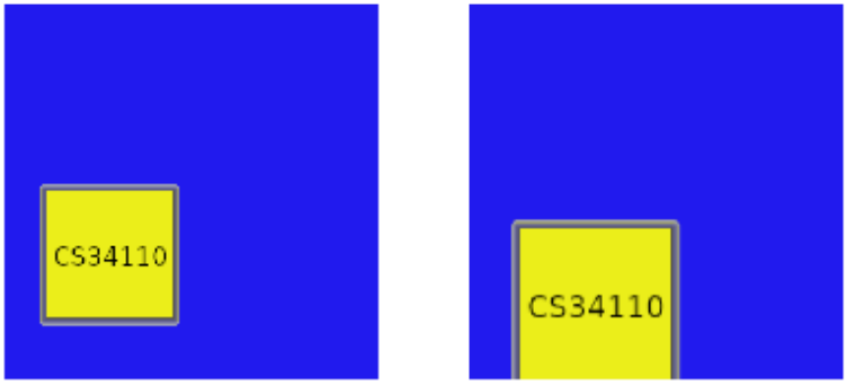
$s_y = 0.5$

$s_x = 0.5; s_y = 0.5$

Scaling properties

Scaling preserves

- Lines
- Angles
- Orientation



Translation

- Algebraically

For each (x,y) $\in \Omega_I$

$$\acute{x} = x + t_x$$

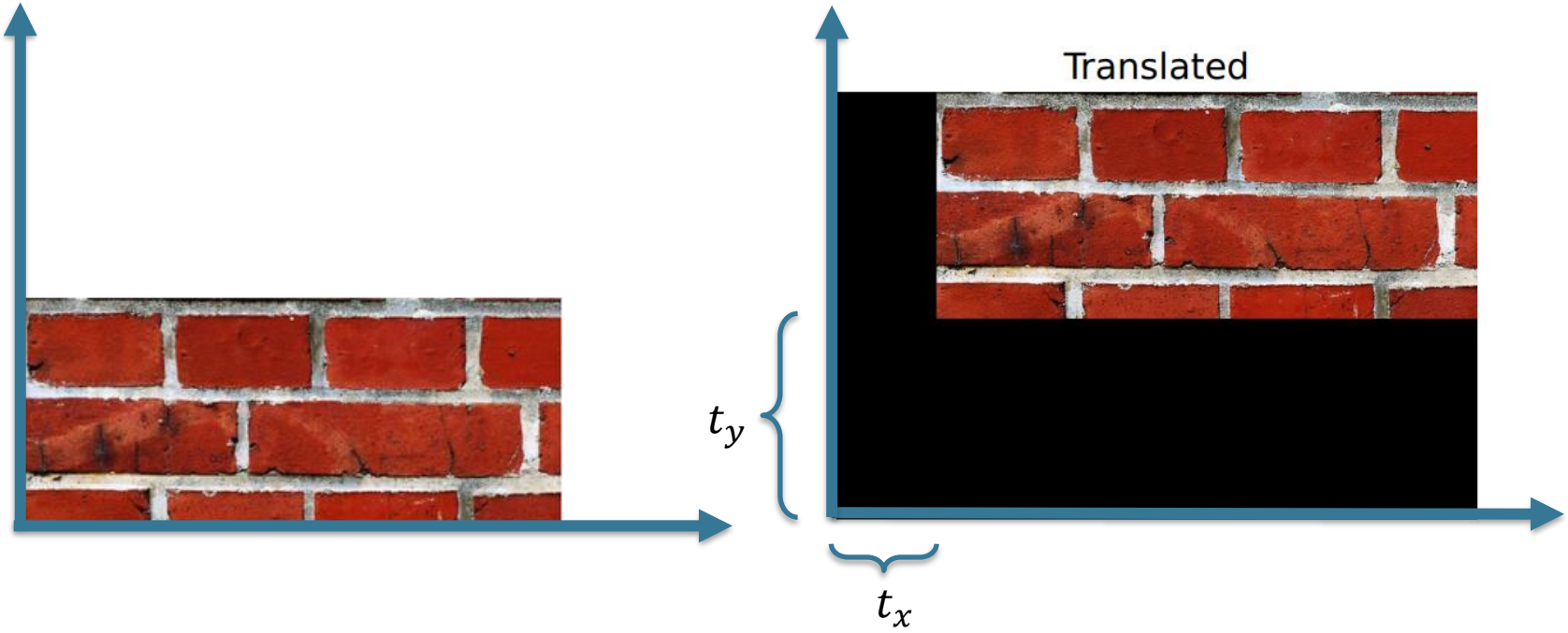
$$\acute{y} = y + t_y$$

$$I(\acute{x}, \acute{y}) = I(x, y)$$

As matrix multiplication:

$$\begin{bmatrix} \acute{x} \\ \acute{y} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix}}_{T_{translation}} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

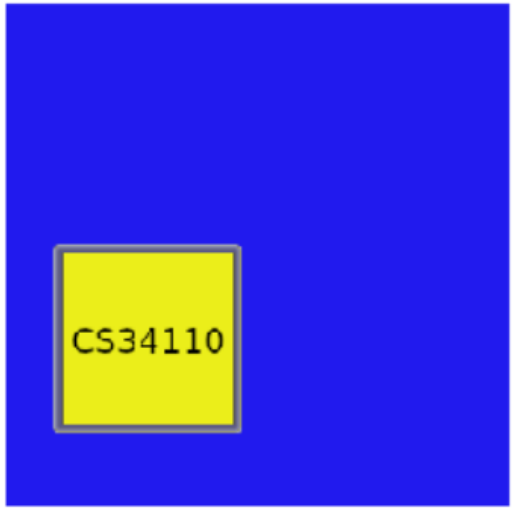
Translating

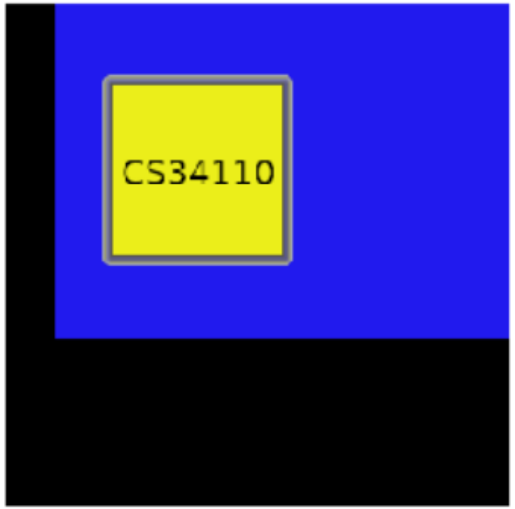


Translation properties

Translation preserves:

- Lines
- Length
- Angles
- Orientation





Rotation

- Algebraically

For each (x,y) $\in \Omega_I$

$$\acute{x} = x\ cos(\theta) - y\ sin(\theta)$$

$$\acute{y} = x\ sin(\theta) + y\ cos(\theta)$$

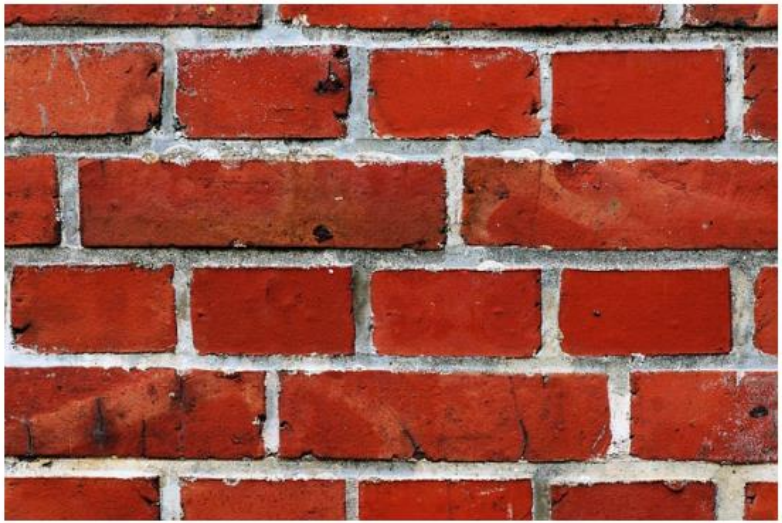
As matrix multiplication:

$$\begin{bmatrix} \acute{x} \\ \acute{y} \end{bmatrix} = \underbrace{\begin{bmatrix} cos(\theta) & -sin(\theta) \\ sin(\theta) & cos(\theta) \end{bmatrix}}_{T_{rotation}} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$I(\acute{x}, \acute{y}) = I(x, y)$$

Rotating

Original

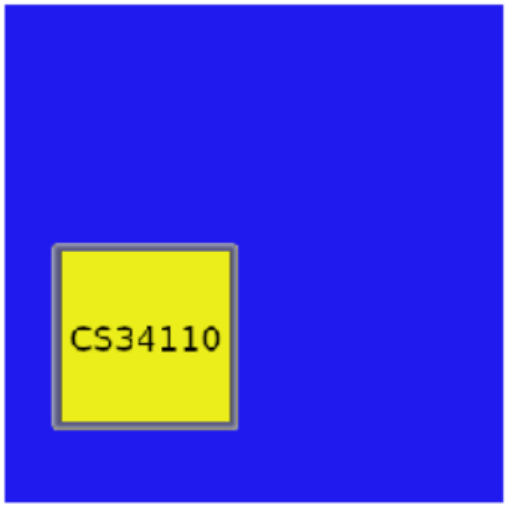


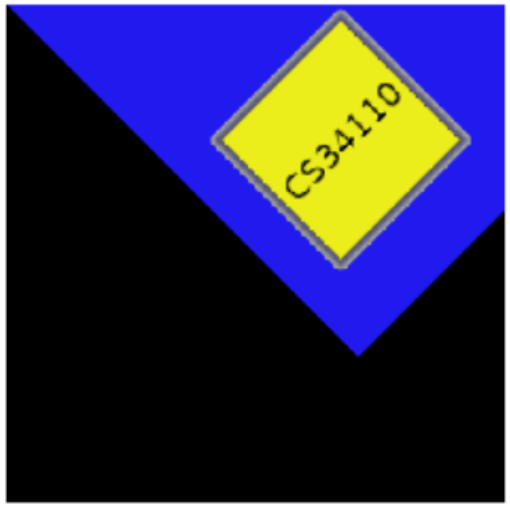
Rotated

Rotation properties

Rotation preserves:

- Lines
- Length
- Angles
- Areas





Note the centre of rotation!

Rigid body transformation (AKA Euclidean)

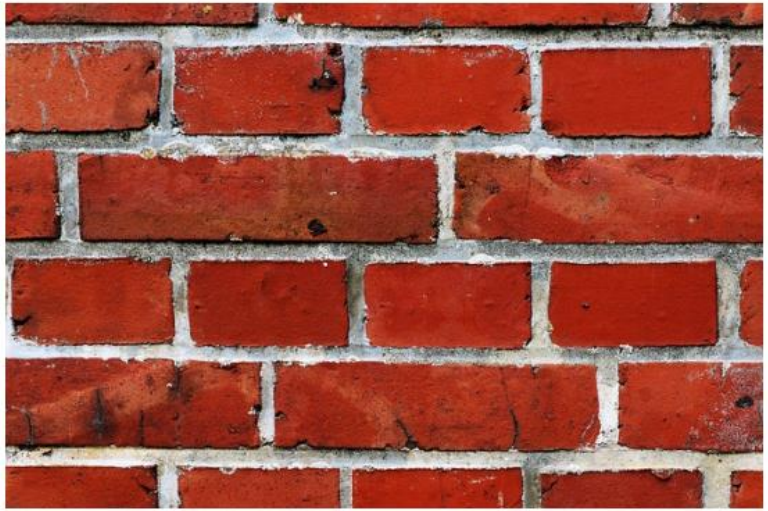
$$T_{Euclidean} = \{T_{rotation}, T_{translation}\}$$

$$\begin{bmatrix} \acute{x} \\ \acute{y} \end{bmatrix} = \underbrace{\begin{bmatrix} cos(\theta) & -sin(\theta) \\ sin(\theta) & cos(\theta) \end{bmatrix}}_{T_{rotation}} \begin{bmatrix} x \\ y \end{bmatrix} + \underbrace{\begin{bmatrix} t_x \\ t_y \end{bmatrix}}_{T_{translation}}$$

$$\begin{bmatrix} \acute{x} \\ \acute{y} \end{bmatrix} = \begin{bmatrix} cos(\theta) & -sin(\theta) & t_x \\ sin(\theta) & cos(\theta) & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Rigid body transforming

Original

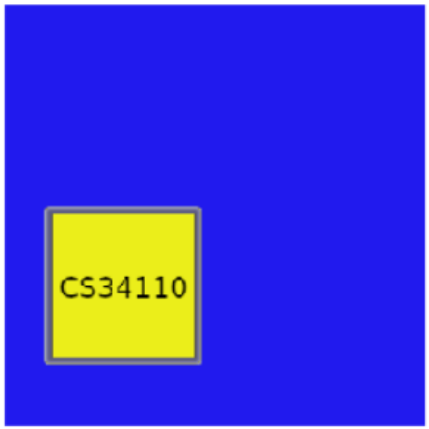


Transformed

Euclidean transformation properties

Rotation preserves:

- Lines
- Length
- Angles
- Areas



$$\begin{bmatrix} \acute{x} \\ \acute{y} \end{bmatrix} = \begin{bmatrix} cos(\theta) & -sin(\theta) & t_x \\ sin(\theta) & cos(\theta) & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Similarity transformation

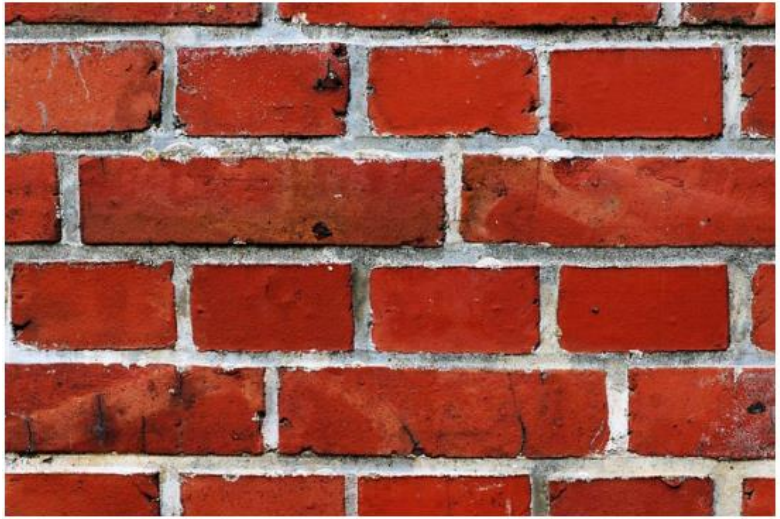
$$T_{similarity} = \{T_{scale}, T_{rotation}, T_{translation}\}$$

$$\begin{bmatrix} \acute{x} \\ \acute{y} \end{bmatrix} = \underbrace{\begin{bmatrix} S & 0 \\ 0 & S \end{bmatrix}}_{T_{scale}} \underbrace{\begin{bmatrix} cos(\theta) & -sin(\theta) \\ sin(\theta) & cos(\theta) \end{bmatrix}}_{T_{rotation}} \begin{bmatrix} x \\ y \end{bmatrix} + \underbrace{\begin{bmatrix} t_x \\ t_y \end{bmatrix}}_{T_{translation}}$$

$$\begin{bmatrix} \acute{x} \\ \acute{y} \end{bmatrix} = \begin{bmatrix} S * cos(\theta) & -S * sin(\theta) & t_x \\ S * sin(\theta) & S * cos(\theta) & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Similarity transforming

Original

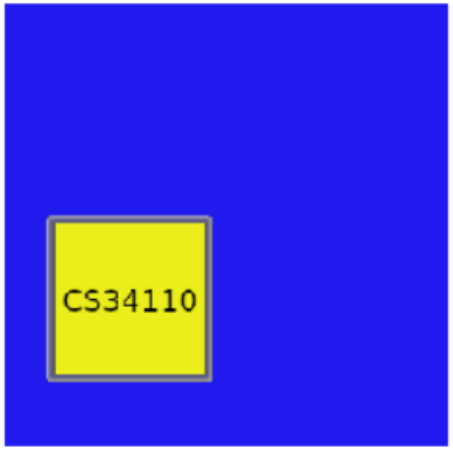


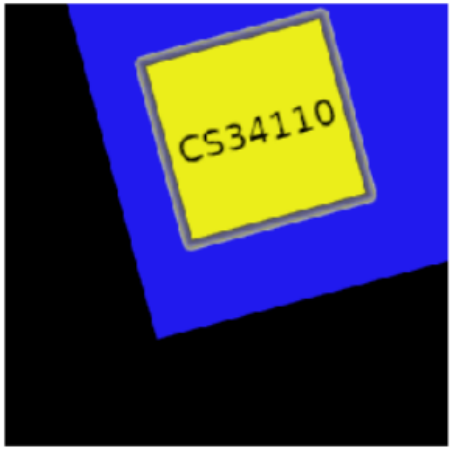
Transformed

Similarity transformation properties

$T_{similarity}$ preserves:

- Lines
- Angles





$$\begin{bmatrix} \acute{x} \\ \acute{y} \end{bmatrix} = \begin{bmatrix} S * cos(\theta) & -S * sin(\theta) & t_x \\ S * sin(\theta) & S * cos(\theta) & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Affine transformation

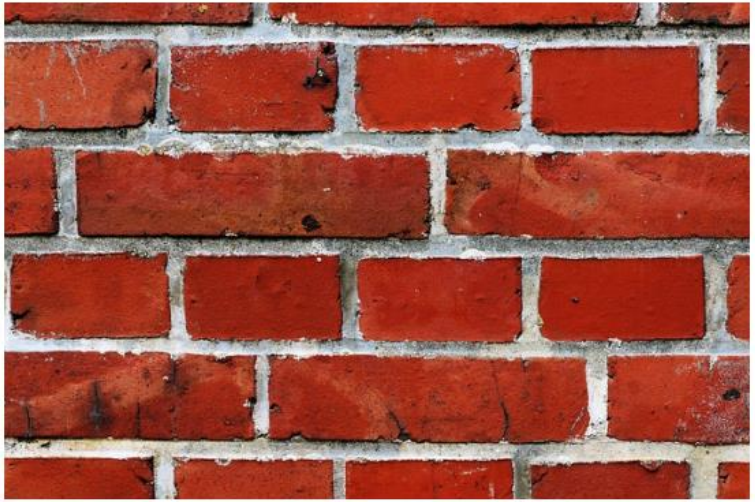
$$\begin{bmatrix} \acute{x} \\ \acute{y} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Keep the result **homogeneous**:

$$\begin{bmatrix} \acute{x} \\ \acute{y} \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Affine transforming

Original

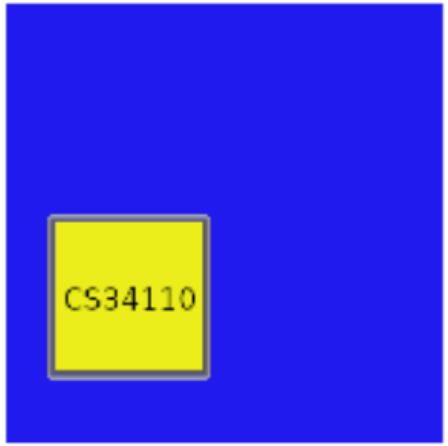


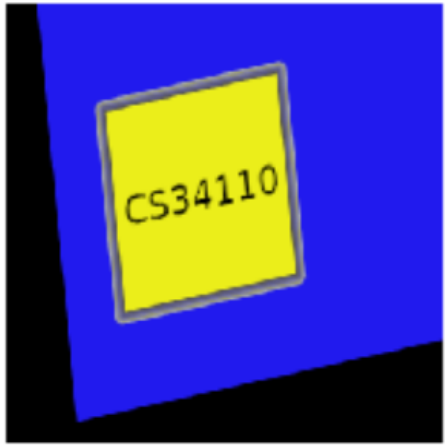
Transformed

Affine transformation properties

T_{affine} preserves

- Lines
- Line parallelism
- Area ratios





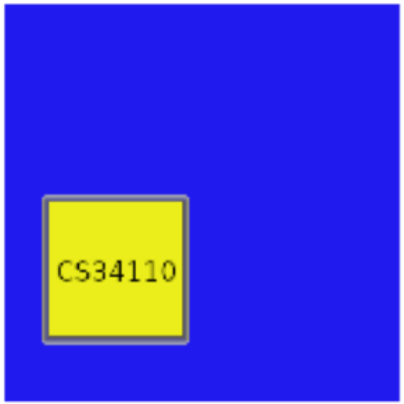
$$\begin{bmatrix} \acute{x} \\ \acute{y} \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

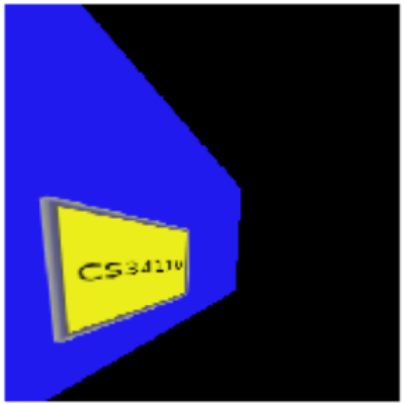
General projective transformation

- for any non-zero constant w

$$\begin{bmatrix} \acute{x} \\ \acute{y} \\ 1 \end{bmatrix} \approx \begin{bmatrix} w\acute{x} \\ w\acute{y} \\ w \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

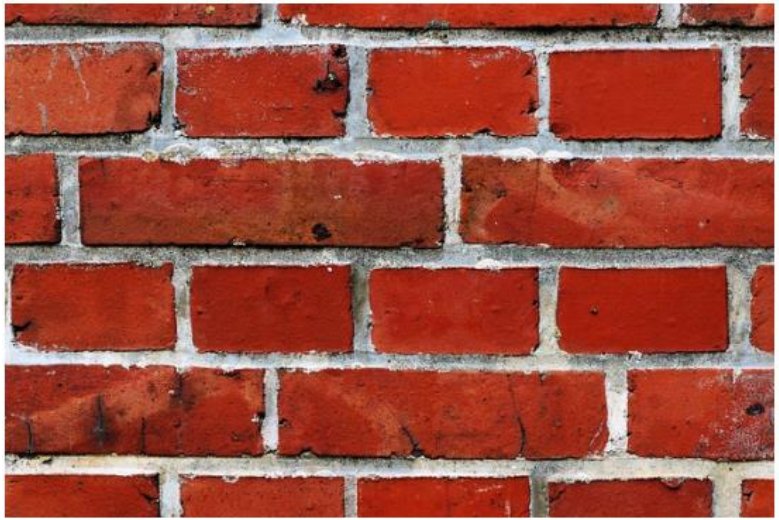
Preserves
- Lines



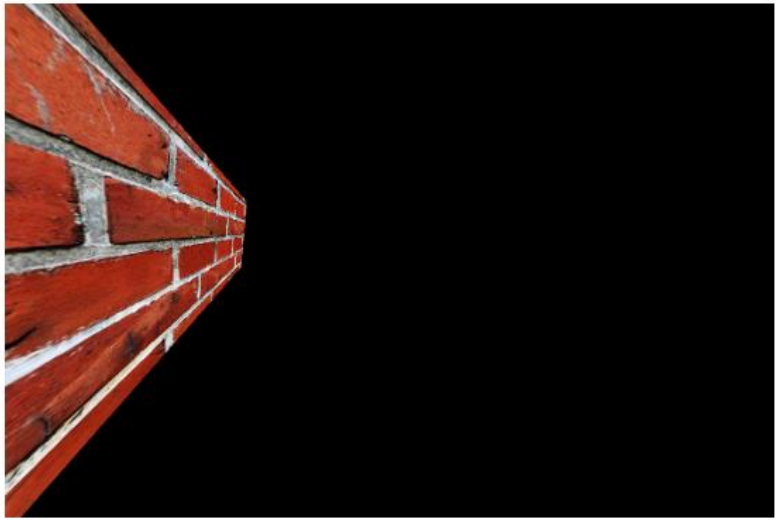


Projective transforming

Original



Transformed



Summary of transformations:

- Euclidean and similarity matrices are easy to build:

$$\begin{bmatrix} \acute{x} \\ \acute{y} \\ 1 \end{bmatrix} = \begin{bmatrix} S * cos(\theta) & -S * sin(\theta) & t_x \\ S * sin(\theta) & S * cos(\theta) & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

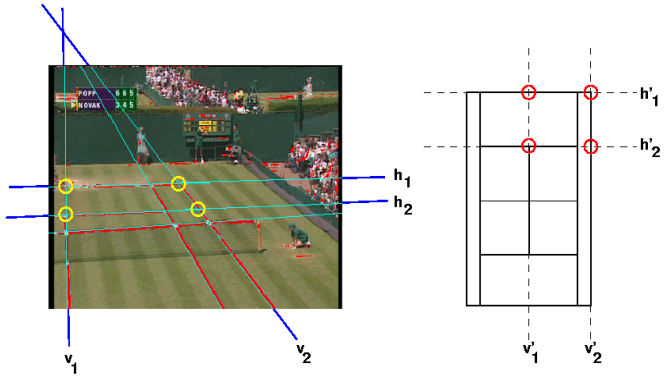
- Affine and projective matrices are easy to build:

$$w \begin{bmatrix} \acute{x} \\ \acute{y} \\ 1 \end{bmatrix} = \begin{bmatrix} w\acute{x} \\ w\acute{y} \\ w \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Matrix from point correspondences

$$w\begin{bmatrix}\acute{x}\\ \acute{y}\\ 1\end{bmatrix}=\begin{bmatrix}w\acute{x}\\ w\acute{y}\\ w\end{bmatrix}=\begin{bmatrix}h_{11} & h_{12} & h_{13}\\ h_{21} & h_{22} & h_{23}\\ h_{31} & h_{32} & 1\end{bmatrix}\begin{bmatrix}x\\ y\\ 1\end{bmatrix}$$

Given (x, y) and $(\acute{x}, \acute{y})$, how to obtain H ?



4 corresponding points

4 correspondences = 8 equations

$$\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 & -x'_1 x_1 & -x'_1 y_1 \\ 0 & 0 & 0 & x_1 & y_1 & 1 & -y'_1 x_1 & -y'_1 y_1 \\ x_2 & y_2 & 1 & 0 & 0 & 0 & -x'_2 x_2 & -x'_2 y_2 \\ 0 & 0 & 0 & x_2 & y_2 & 1 & -y'_2 x_2 & -y'_2 y_2 \\ x_3 & y_3 & 1 & 0 & 0 & 0 & -x'_3 x_3 & -x'_3 y_3 \\ 0 & 0 & 0 & x_3 & y_3 & 1 & -y'_3 x_3 & -y'_3 y_3 \\ x_4 & y_4 & 1 & 0 & 0 & 0 & -x'_4 x_4 & -x'_4 y_4 \\ 0 & 0 & 0 & x_4 & y_4 & 1 & -y'_4 x_4 & -y'_4 y_4 \end{bmatrix} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{12} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \end{bmatrix} = \begin{bmatrix} x'_1 \\ y'_1 \\ x'_2 \\ y'_2 \\ x'_3 \\ y'_3 \\ x'_4 \\ y'_4 \end{bmatrix}$$

4 corresponding points

Which is of the form:

$$AH = b$$

And can be solved by matrix inversion:

$$A^{-1}AH = A^{-1}b$$

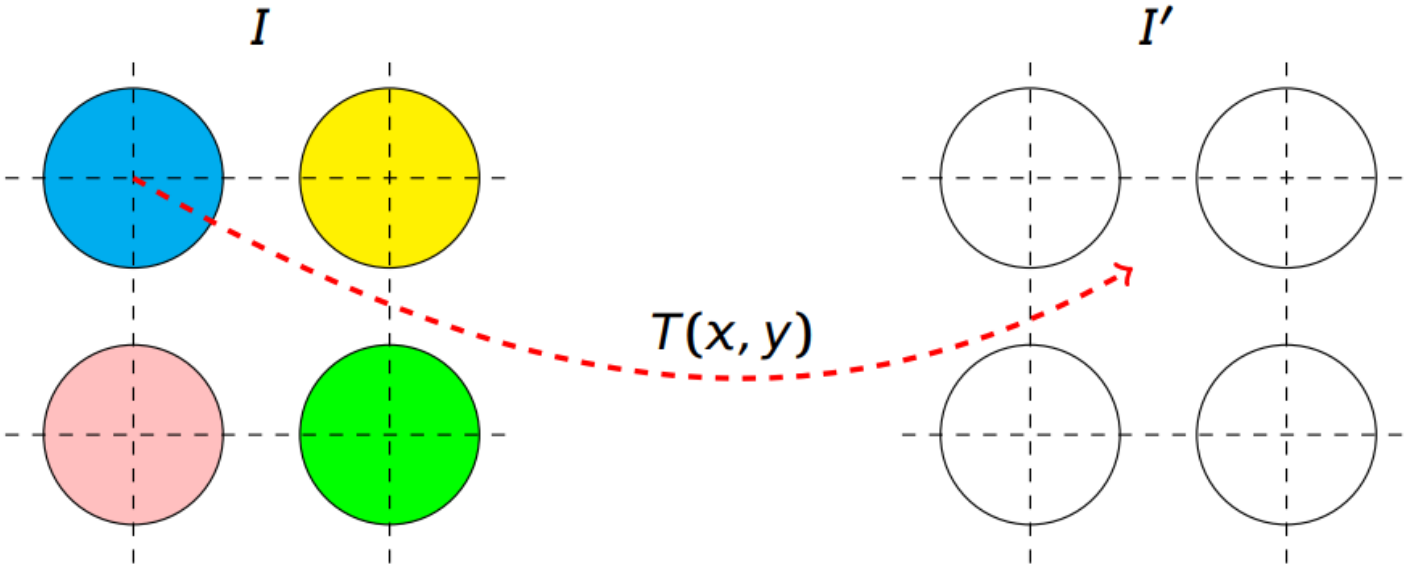
More corresponding points

- We can have more point correspondences ($n > 4$):

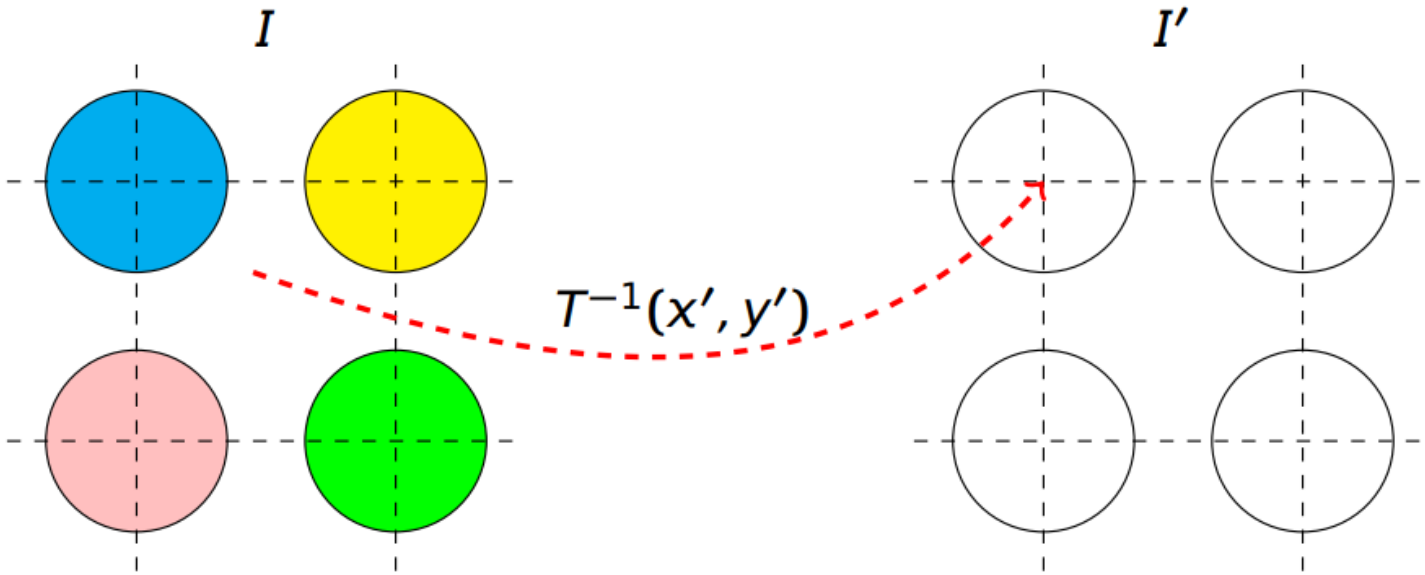
$$\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 & -x'_1x_1 & -x'_1y_1 \\ 0 & 0 & 0 & x_1 & y_1 & 1 & -y'_1x_1 & -y'_1y_1 \\ x_2 & y_2 & 1 & 0 & 0 & 0 & -x'_2x_2 & -x'_2y_2 \\ 0 & 0 & 0 & x_2 & y_2 & 1 & -y'_2x_2 & -y'_2y_2 \\ & & & & \vdots & & & \\ x_n & y_n & 1 & 0 & 0 & 0 & -x'_nx_n & -x'_ny_n \\ 0 & 0 & 0 & x_n & y_n & 1 & -y'_nx_n & -y'_ny_n \end{bmatrix} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{12} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \end{bmatrix} = \begin{bmatrix} x'_1 \\ y'_1 \\ x'_2 \\ y'_2 \\ \vdots \\ x'_n \\ y'_n \end{bmatrix}$$

It can be solved by minimizing: $\frac{1}{2}||AH - b||^2$

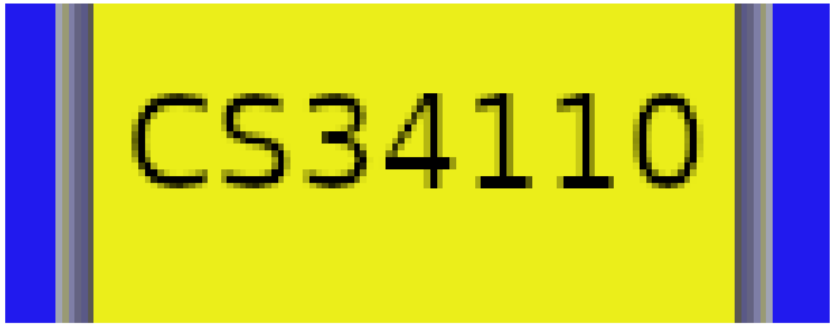
Forward Wrapping

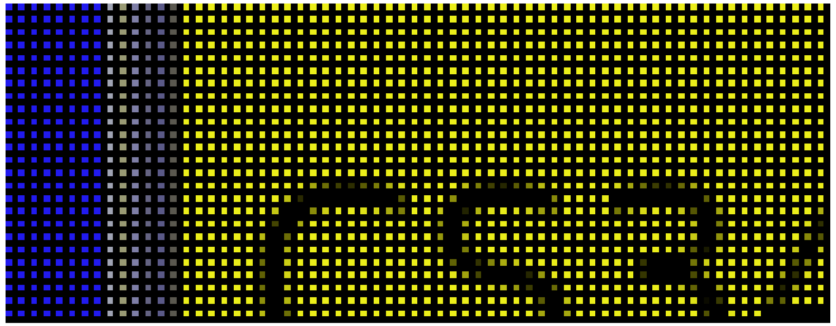


Inverse Wrapping



Example: Scaling 2***x*** with forward warping:

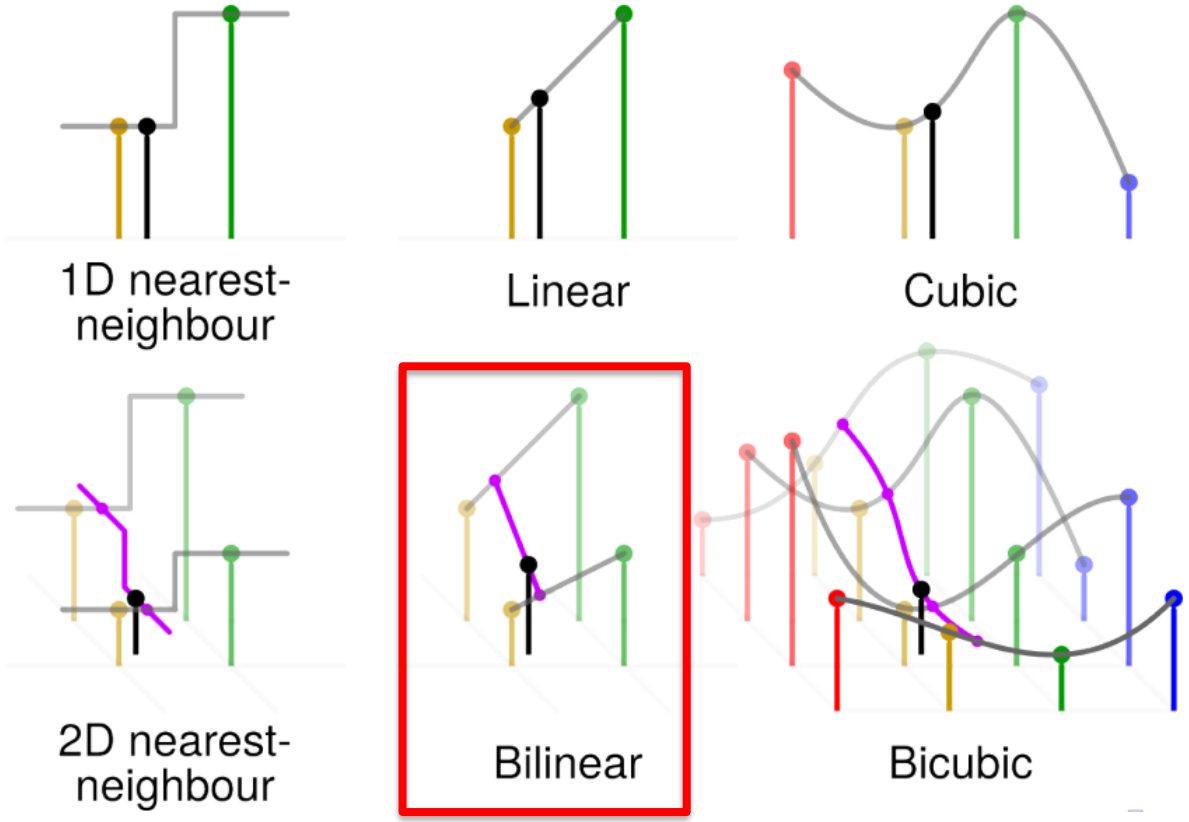




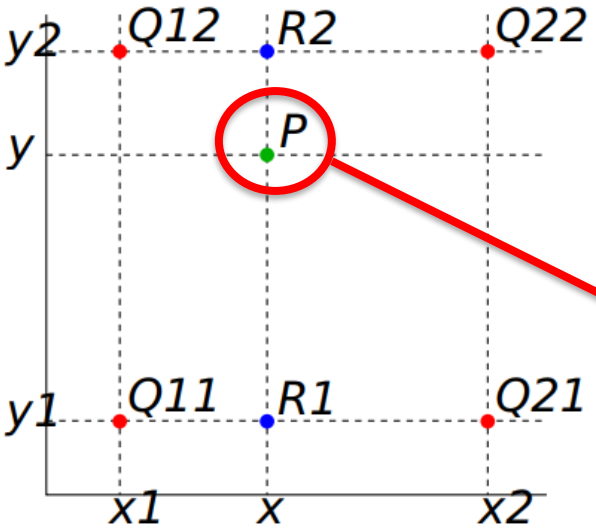
$$(x', y') = T(x, y)$$

Remember: $I'(x', y') \leftarrow I(x, y)$

Interpolation



Bilinear Interpolation



Credit: Wikipedia

$$R_1 \approx \frac{x_2 - x}{x_2 - x_1} Q_{11} + \frac{x - x_1}{x_2 - x_1} Q_{21}$$

$$R_2 \approx \frac{x_2 - x}{x_2 - x_1} Q_{12} + \frac{x - x_1}{x_2 - x_1} Q_{22}$$

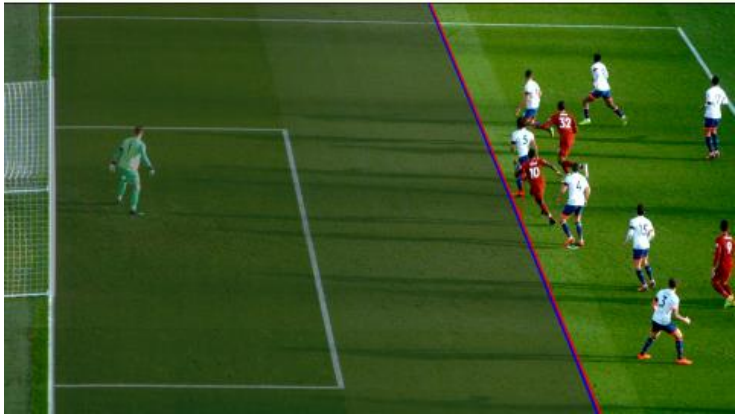
$$P \approx \frac{y_2 - y}{y_2 - y_1} R_1 + \frac{y - y_1}{y_2 - y_1} R_2$$

Can we “look” at the walls as if we were in front of them?

Yes, with a homography

More uses

- Augmented reality
- Virtual adverts:
  https://youtu.be/dMLGp_NRgVQ



Forward Wrapping

Panorama with homography

Stitching a panorama

1) Take sequence of images from the same position (each with camera slightly rotated about its optical centre)
2) Compute homography between I_1 and I_2 (or vice-versa)
3) Transform I_1 to overlap I_2
4) Blend
5) Repeat (previous steps) if there are more images

Morphing

- Transforming one image into another
- Also involves geometric transforms
- Check:
  https://youtu.be/CCvp_E9jMsI

Credit: Beier and Neely

Review

- Geometric image transformations can be specified by a matrix
- We can “manually” define the parameters of the matrix
- For complex transformations we need to estimate the parameters
- Forward transform is the wrong way to go
- Backward transform is the way to go, but requires interpolation
- When downscaling high frequency images, we might need to pre-smooth to avoid aliasing